

Quelques mathématiques derrière `elbow-helper`

Warith Harchaoui

Été 2026

Résumé

Cette note explique, à partir des principes de base, chaque brique mathématique sur laquelle repose `elbow-helper` : le pipeline à coude unique livré (`robust_knee`, `robust_elbow`), la recherche multi-coudes qui alimente `robust_knees` et le socle commun de vraisemblance gaussienne sur lequel repose chaque critère de sélection de modèle du paquet. Les références se trouvent dans `references.bib`. Pour chaque affirmation, on précise s'il s'agit d'une citation vérifiée, d'une citation rapportée par une source secondaire, ou d'une construction propre à ce projet, selon la convention de sourçage adoptée dans l'ensemble du dépôt.

Comment lire cette note. Chaque notion reçoit le même traitement : d'abord une explication en langage ordinaire, appuyée sur un exemple concret et ne supposant rien au-delà de l'algèbre de lycée, puis la formule précise, pour qui veut vérifier chaque étape, y compris un lecteur titulaire d'un doctorat en mathématiques appliquées qui préfère contrôler la dérivation plutôt que faire confiance au texte. Les blocs de formules peuvent être sautés en première lecture si l'intuition suffit déjà à répondre à la question, quitte à y revenir plus tard.

Une introduction dérive, une seule fois, la machinerie qui va de la vraisemblance brute à un score borné et que chaque critère ultérieur réutilise : pourquoi BIC, mBIC et ICL reposent tous sur le logarithme naturel, et comment un pire cas délibérément pessimiste, plutôt qu'un nombre brut non borné, transforme n'importe lequel d'entre eux en un score $[0, 1]$ justifié et interprétable. À partir de là, toute la note tourne autour d'une seule question posée trois fois, avec un réalisme croissant : que signifie géométriquement *un* coude (partie [II](#)), que signifie géométriquement *plusieurs* coudes (partie [III](#)) et comment faire confiance à l'une ou l'autre réponse une fois les données bruitées (partie [IV](#)) ?

Table des matières

I	Introduction	4
1	De la vraisemblance brute à un score borné	4
2	Travaux connexes	6
II	Un coude, la géométrie d'une pliure	10
3	Mettre les données sur un pied d'égalité : la normalisation	10
4	La bosse de la courbe de différence : ce que « coude » veut dire, précisément	10
5	Le modèle en ligne brisée : un coude comme changement de pente	11
6	Un exemple travaillé : le coude d'inertie du k-means	11
III	Plusieurs coudes, la géométrie de plusieurs pliures	13
7	Le modèle : des segments linéaires indépendants	13
8	Le coût d'un segment, sous forme close	13
9	Le partitionnement optimal : la programmation dynamique	14
10	La segmentation binaire gloutonne : l'alternative plus rapide, moins bonne	14
11	Un exemple travaillé : une montagne aux pentes alternées	17
IV	Un ou plusieurs coudes, avec du bruit	17
12	Y a-t-il seulement une tendance ? La corrélation de rang de Spearman	18
13	Ne pas se fier à une seule échelle : la recherche lissage $\times$ sensibilité	19
14	Ne le compter que s'il survit partout : le regroupement par persistance	19
15	Confirmer la pente : le contraste de Theil-Sen	19
16	Confirmer le modèle : le BIC et la validation croisée par blocs	20
17	Le bootstrap : le coude survit-il à une reprise ?	21
18	Le test nul : une droite pourrait-elle expliquer cela par hasard ?	21
19	Le BIC nu pour plusieurs coudes : le critère naïf et pourquoi il surestime	22

20 Le BIC modifié : la correction de Zhang et Siegmund et une convention de signe résolue par le test	22
21 L'ICL : des modèles de mélange aux points de rupture, avec un <i>bug</i> trouvé et corrigé au passage	23
22 Un test séquentiel à taux d'erreur par famille contrôlé	24
23 Ce que cela laisse à l'API livrée	25
24 Un exemple travaillé : un escalier de sigmoïdes	25
25 Un exemple travaillé : le même coude discret, à deux niveaux de bruit	25
26 Un exemple travaillé : le diagramme des éboulis de l'ACP	26
27 Un exemple travaillé : une courbe de saturation exponentielle	27
28 Un exemple travaillé : latence de file d'attente et coude de dimensionnement	28
29 Un exemple travaillé : dimensionnement de cache et rendements décroissants	28
30 Un exemple travaillé : quand arrêter les tests	30

Première partie

Introduction

Sur une courbe à rendements décroissants, un coude marque l'endroit où chaque unité supplémentaire investie rapporte de moins en moins. Les détecteurs existants savent bien proposer un point ; ce qui leur manque est une notion de confiance, si bien qu'une estimation ponctuelle isolée sur une courbe bruitée est facile à surinterpréter. `elbow-helper` enveloppe un repéreur réécrit de zéro dans une procédure de décision prudente et refuse de trancher franchement quand les preuves sont trop minces.

Cette note dérive, à partir des principes de base, chaque brique mathématique que le paquet met en œuvre. Plusieurs de ses critères de sélection de modèle, des scores qui récompensent un bon ajustement tout en pénalisant la complexité superflue, remontent à la même log-vraisemblance gaussienne : le BIC à coude unique, le critère d'information bayésien (section 16) ; le BIC(k) à plusieurs coudes de la section 19 ; son cousin plus strict, le BIC modifié, mBIC (section 20) ; et une version qui facture aussi l'ambiguïté des frontières de segments, la vraisemblance complétée intégrée, ICL (section 21). Cette introduction dérive ce socle commun une seule fois, en logarithme naturel de bout en bout, avant qu'aucune de ces sections n'y ajoute son propre terme de pénalité.

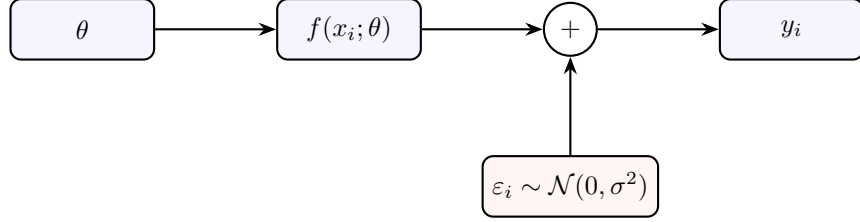
1 De la vraisemblance brute à un score borné

La notion, en bref. La *vraisemblance* désigne habituellement la densité jointe des données observées, lue comme une fonction du paramètre plutôt que des données, $L_{\text{brute}}(\theta) := \prod_{i=1}^n p(y_i | x_i, \theta)$. Cette note prend au contraire la vraisemblance *par observation* dès le départ, une moyenne *exponentiée* de log-densités plutôt qu'un produit brut,

$$L(\theta, \sigma^2) := \exp(\mathbb{E}[\ln p]) = L_{\text{brute}}^{1/n},$$

la moyenne géométrique des n densités individuelles, exactement la racine n -ième de L_{brute} . Cet écart par rapport à la définition classique, et pourquoi il est principiel plutôt qu'une simple remise à l'échelle de confort, fait l'objet d'une note compagne, `LIKELIHOOD-fr.tex`, séparée parce que ce socle est général et ne dépend en rien de l'ajustement de courbes : l'inégalité de Jensen et un raffinement indexé par une température emprunté à la mécanique statistique à l'équilibre, les grandes déviations de Cramér et la martingale de Wald, un compte rendu précis de L comme estimateur statistique avec son biais et sa variance dérivés et vérifiés par simulation, et comment cette même construction se lit sur un modèle de classification catégorielle (avec sa propre simulation travaillée) aussi bien que sur le modèle de régression gaussienne dérivé plus bas. Cette note construit directement dessus : chaque critère de la famille BIC ci-dessous hérite de l'instanciation gaussienne de L . Le test séquentiel fondé sur la permutation de la section 22 est exactement le cas que `LIKELIHOOD-fr.tex` identifie où la borne martingale classique de Wald ne s'applique pas directement, une alternative composite plutôt qu'entièrement spécifiée à chaque étape.

Sous un modèle génératif $y_i = f(x_i; \theta) + \varepsilon_i$ avec ε_i i.i.d. $\mathcal{N}(0, \sigma^2)$ (f n'importe lequel des modèles de courbe que cette note ajuste, une droite simple, une ligne brisée, ou une ligne brisée à k segments ; θ ses paramètres de forme) :



$$L(\theta, \sigma^2) := \left(\prod_{i=1}^n (2\pi\sigma^2)^{-1/2} \exp\left(-\frac{(y_i - f(x_i; \theta))^2}{2\sigma^2}\right) \right)^{1/n} \quad (1)$$

Le produit de n préfacteurs identiques et le produit d'exponentielles s'effondrent proprement sous la racine $1/n$: un produit d'exponentielles est une exponentielle d'une somme, et cette somme divisée par n est une moyenne.

$$L(\theta, \sigma^2) = (2\pi\sigma^2)^{-1/2} \exp\left(-\frac{1}{2\sigma^2} \cdot \frac{1}{n} \sum_{i=1}^n (y_i - f(x_i; \theta))^2\right) = (2\pi\sigma^2)^{-1/2} \exp\left(-\frac{\text{MSE}(\theta)}{2\sigma^2}\right) \quad (2)$$

l'erreur quadratique moyenne par point, $\text{MSE}(\theta) := \frac{1}{n} \sum_i (y_i - f(x_i; \theta))^2$. L est déjà une expression en exp, en base e , directement issue de la définition même de la densité gaussienne, avant qu'un analyste choisisse de prendre un logarithme par commodité. Prendre le $\ln$ de L n'introduit donc pas de base : il défait celle qui est déjà là :

$$\ell(\theta, \sigma^2) := \ln L(\theta, \sigma^2) = -\frac{1}{2} \ln(2\pi\sigma^2) - \frac{\text{MSE}(\theta)}{2\sigma^2} \quad (3)$$

Minimiser MSE sur θ à σ^2 fixé revient à maximiser ℓ sur θ : c'est pourquoi les moindres carrés ordinaires et le maximum de vraisemblance gaussien forment une seule et même procédure. En profilant σ^2 à sa valeur la mieux soutenue par les données, l'estimateur du maximum de vraisemblance (EMV) $\hat{\sigma}^2 = \text{MSE}$, puis en exponentiant -2ℓ , l'échelle conventionnelle d'un critère d'information, on défait le $\ln$ et on obtient une seule quantité limpide par observation, la mesure que la théorie de l'information appelle perplexité :

$$\text{PPL}(\theta) := \exp(-2\ell(\theta, \hat{\sigma}^2)) = 2\pi e \cdot \text{MSE}(\theta) \quad (4)$$

En remontant cette perplexité par observation à l'échelle de l'échantillon complet à n points, on retrouve la log-vraisemblance négative, NLL, dont part toute formule de la famille BIC dans cette note, écrite ici via la somme des carrés des résidus, $RSS(\theta) := \sum_i (y_i - f(x_i; \theta))^2$, le numérateur non moyenné que MSE divise par n :

$$\text{NLL} := n \ln \text{PPL}(\theta) = n \ln\left(\frac{RSS}{n}\right) + n(1 + \ln 2\pi) \quad (5)$$

le même terme $n \ln(RSS/n)$ par lequel s'ouvre toute formule de la famille BIC dans cette note, mesuré en *nats* : l'analogue en logarithme naturel du bit, un nat est l'information gagnée par un événement $1/e$ fois moins probable que la certitude, l'unité dans laquelle toute quantité construite avec $\ln$ plutôt que $\log_2$ s'exprime implicitement. Chaque log-vraisemblance, chaque NLL, BIC, mBIC et ICL dans cette note est un compte de nats, pas un nombre sans unité, ce qui explique en partie pourquoi une valeur brute comme « 282,4 nats » ne se lit pas comme un score borné : les

nats s'accumulent avec n et avec l'échelle propre de la log-vraisemblance, exactement la motivation derrière les scores bornés dérivés plus bas.

Cas extrêmes. Le PPL est une amélioration véritable par rapport au NLL brut dont il est issu, sur exactement un côté : le NLL parcourt $(-\infty, +\infty)$, mais exponentier corrige automatiquement la queue inférieure, $\text{PPL} = 2\pi e \cdot \text{MSE}(\theta) \geq 0$ toujours, avec $\text{PPL} = 0$ à l'ajustement parfait ($\text{MSE} \rightarrow 0$) : la dégénérescence gaussienne qui justifie l'existence même du BIC, un modèle avec autant de paramètres que de points pousse le MSE vers zéro et sa vraisemblance brute vers l'infini. Le côté supérieur reste non résolu : $\text{MSE} \rightarrow \infty$ n'est pas un pire cas significatif ; il est simplement non borné, pas un vrai point de référence près duquel un ajustement se situerait. La moyenne d'échantillon $\bar{y}$ n'est pas non plus une bonne référence de pire cas : c'est l'*unique* constante qui *minimise* le MSE parmi toutes les constantes, le meilleur prédicteur trivial qui soit, pas un mauvais. C'est exactement pourquoi normaliser contre elle (le R^2 classique) peut encore donner un résultat négatif : un ajustement réel est facile à rendre pire que la seule constante que rien d'autre ne peut battre. Un pire cas vraiment pessimiste et fini choisit plutôt le point *observé* le plus difficile à partir duquel prédire tous les autres :

$$y_{\text{mauvais}} := \arg \max_{y_j \in \{y_1, \dots, y_n\}} \frac{1}{n} \sum_i (y_i - y_j)^2, \quad \text{MSE}_{\text{pire}} := \frac{1}{n} \sum_i (y_i - y_{\text{mauvais}})^2 \quad (6)$$

Comme $\bar{y}$ minimise le MSE de façon unique parmi les constantes, toute autre constante, une valeur effectivement observée en premier lieu, obtient un score strictement pire : $\text{MSE}_{\text{pire}} > \text{Var}(y)$ toujours. Normaliser contre ce plafond plus difficile à battre donne à la place

$$1 - \frac{\text{MSE}(\theta)}{\text{MSE}_{\text{pire}}} = 1 - \frac{\text{PPL}(\theta)}{\text{PPL}_{\text{pire}}} \quad (7)$$

un score ancré à 1 pour un ajustement parfait et à 0 pour un ajustement aussi mauvais que y_{mauvais} lui-même, avec une barre bien plus haute à franchir pour tomber en territoire négatif que ce que le R^2 fondé sur la moyenne offre jamais, puisque $\text{MSE}_{\text{pire}} > \text{Var}(y)$ par construction, construit entièrement à partir du MSE contre un point de référence réel et délibérément pessimiste, sans facteur de Bayes ni probabilité a posteriori nulle part dans la dérivation. C'est un cousin plus strict du R^2 , pas le R^2 lui-même : le R^2 garde $\bar{y}$ comme référence et accepte en échange un score parfois négatif. La légende de diagnostic d'`elbow_helper.plotting` prend une autre voie : elle convertit le ΔBIC directement en probabilité a posteriori via un facteur de Bayes (Kass and Raftery, 1995), une normalisation alternative tout aussi valable, construite sur le même NLL mais différente de celle dérivée ici.

Toute quantité de la famille BIC dans cette note hérite de cette même base naturelle par construction : chacune est le NLL plus une pénalité de complexité, jamais une vraisemblance redérivée depuis zéro. Changer la base du logarithme n'importe où en aval reviendrait à redériver la densité gaussienne elle-même dans cette base, pas à redimensionner un nombre déjà obtenu.

2 Travaux connexes

Heuristiques de coude et d'elbow. La littérature sur la détection de coude a un point de référence dominant : le Kneedle de Satopaa, Albrecht, Irwin et Raghavan (Satopaa et al., 2011), construit exactement pour le cadre par lequel cette note s'ouvre, un opérateur système face à une courbe qui a besoin d'une réponse automatique à « où cela plie-t-il ». Le mécanisme de Kneedle est la même géométrie de distance à la corde que la section 4 construit : normaliser la courbe au carré unité, soustraire la droite reliant ses deux extrémités et appeler coude le point de distance verticale

maximale à cette corde. C’est rapide, peu paramétré, et, sur une courbe propre, généralement juste. Ce que cela ne fait pas, c’est dire à quel point c’est juste : Kneedle renvoie un seul indice, que la courbe soit un coude manuel ou trois points de bruit pur qui se trouvent osciller dans le bon sens, sans aucune notion de la façon dont cette réponse changerait sous un rééchantillonnage, sous une autre échelle de lissage ou face à un modèle nul sans coude du tout. Cet écart, une estimation ponctuelle sans confiance attachée, est précisément ce que cette note entière existe à combler. Le repéreur d’**elbow-helper** garde l’intuition géométrique de Kneedle en son cœur, la courbe de différence normalisée reste l’origine de chaque coude candidat, mais traite ce candidat comme une hypothèse à mettre à l’épreuve (sections 13–18) plutôt que comme une réponse à renvoyer telle quelle.

Détection de points de rupture et segmentation. Une fois la question passée d’un coude à plusieurs, la littérature pertinente n’est plus la détection de coude mais la détection de points de rupture. Elle se scinde selon une ligne coût contre flexibilité que cette note doit trancher : une recherche exacte qui ne vaut son coût que jusqu’à un $k_{\max}$ borné ou une approximation gloutonne qui passe mieux à l’échelle mais peut s’enfermer dans une erreur précoce. L’algorithme de programmation dynamique élaguée de Killick, Fearnhead et Eckley, Pruned Exact Linear Time (PELT) (Killick et al., 2012), se situe du côté exact : un programme dynamique pénalisé, élagué de sorte que les frontières candidates probablement sous-optimales soient écartées avant même d’être évaluées, retrouvant la même segmentation optimale qu’une recherche non élaguée, à coût linéaire plutôt que quadratique, avec k lui-même déterminé par la pénalité plutôt que fixé à l’avance. La section 9 repose sur le même argument de sous-structure optimale, restreint ici à un petit $k_{\max}$ explicitement borné (peu de courbes réelles ont plausiblement plus d’une poignée de vraies pliures) plutôt qu’au k non borné et piloté par pénalité de PELT, ce qui est justement ce qui garde la recherche assez petite pour faire tourner par-dessus les couches de bootstrap et de permutation dont PELT lui-même n’a pas d’équivalent. La segmentation binaire sauvage de Fryzlewicz (Fryzlewicz, 2014) se situe de l’autre côté de cette ligne : une procédure de division récursive et gloutonne sur des sous-intervalles tirés au hasard, plus rapide que la recherche exacte mais susceptible de figer tôt une mauvaise coupure qu’elle ne pourra plus jamais reconsidérer, exactement le mode d’échec que la section 10 démontre sur un exemple construit et la raison pour laquelle ce projet évalue la segmentation gloutonne comme l’alternative plus rapide mais moins bonne, plutôt que comme le choix par défaut. Yao (Yao, 1988) est le premier à relier directement le comptage de points de rupture au critère de Schwarz, l’article dont le $\text{BIC}(k)$ à plusieurs coudes de la section 19 retrace l’ascendance, même si, comme cette section le montre, les hypothèses du BIC nu ne survivent pas intactes au contact d’un emplacement de rupture inconnu.

Critères de sélection de modèle. Chaque critère que cette note construit répond à la même question, la complexité supplémentaire en vaut-elle la peine, avec des notions de moins en moins naïves de ce que « complexité » coûte. Schwarz (Schwarz, 1978) fournit le point de départ : une approximation de Laplace à la vraisemblance marginale d’un modèle, qui facture un $\frac{1}{2} \ln n$ fixe par paramètre libre, correct pour les familles paramétriques régulières mais qui suppose silencieusement absente exactement l’irrégularité qu’introduit l’emplacement d’un point de rupture, une hypothèse dont la section 19 montre l’échec concret, pas seulement en principe. Zhang et Siegmund (Zhang and Siegmund, 2007) diagnostiquent cette même irrégularité et la corrigent par une pénalité sensible à la longueur des segments, le mBIC de la section 20, même si leur propre dérivation laisse une véritable ambiguïté de signe que cette note résout empiriquement plutôt que par hypothèse (encore la section 20). Biernacki, Celeux et Govaert (Biernacki et al., 2000) abordent la complexité sous un angle entièrement différent, développé à l’origine pour choisir le nombre de composantes d’un modèle de mélange, pas du tout un compte de points de rupture, facturant non seulement les paramètres mais aussi l’ambiguïté de l’affectation qui en résulte : deux amas qui se chevauchent et sont difficiles à séparer coûtent plus cher que deux amas propres, même à ajustement égal. Rigai, Lebarbier et

Robin (Rigaill et al., 2012) fournissent la machinerie qui rend praticable le transfert de cette idée aux points de rupture, une distribution a posteriori exacte par passage avant-arrière sur tout l’espace des segmentations discrètes, structurellement la même récurrence que l’algorithme avant-arrière d’un modèle de Markov caché, simplement sur des segmentations plutôt que sur des états cachés, accélérée plus tard par Cleynen, Luong, Rigaill et Nuel (Cleynen et al., 2013). Le récit que fait la section 21 de cette adaptation inclut un *bug* que les tests de ce projet ont attrapé directement : une première version qui laissait le terme de vraisemblance intégrée dominer la correction d’entropie et surestimer sur du bruit pur, un mode d’échec auquel la littérature originale sur les modèles de mélange n’a pas à faire face de la même façon. Kass et Raftery (Kass and Raftery, 1995) se situent légèrement à l’écart de toute cette famille : non pas une pénalité de complexité, mais une lecture décisionnelle d’une différence de log-vraisemblance comme un facteur de Bayes approché, la normalisation alternative fondée sur une sigmoïde que la section 1 dérive puis met de côté au profit du score ancré sur le pire cas utilisé dans le reste de cette note.

Tests séquentiels et multi-échelles. Une troisième famille de méthodes répond directement à la question « ce point de rupture est-il réel », comme un test d’hypothèse plutôt que comme un score pénalisé par la complexité. Davies (Davies, 1977) identifie la difficulté centrale que tout test de ce genre doit résoudre : l’emplacement du point de rupture est un paramètre de nuisance qui n’existe que sous l’hypothèse alternative, si bien que l’asymptotique usuelle du rapport de vraisemblance ne s’applique pas, exactement le problème de pivot que le test séquentiel de la section 22 contourne en utilisant un nul de permutation plutôt qu’asymptotique. Le Simultaneous MUlti-scale Change-point Estimator de Frick, Munk et Sieling (SMUCE) (Frick et al., 2014) résout un problème apparenté mais plus ambitieux, une seule affirmation de confiance qui tient simultanément à chaque échelle de lissage à la fois, sensiblement plus de machinerie que ce dont la section 22 a besoin pour un test séquentiel, un point de rupture à la fois, dont la validité se vérifie directement par simulation plutôt que par la théorie asymptotique. Les deux ont été considérés puis écartés au profit de la construction fondée sur la permutation effectivement livrée, un compromis délibéré : moins de généralité théorique contre une garantie que ce projet a pu vérifier empiriquement plutôt que simplement citer.

Statistique robuste et classique. Le pipeline à coude unique s’appuie sur des outils plus anciens et plus étroits partout où une méthode classique, robuste et bien comprise existe déjà, plutôt que d’en inventer une sur mesure. La corrélation de rang de Spearman (Spearman, 1904) filtre la présence d’une tendance monotone avant toute recherche de coude, choisie plutôt qu’une corrélation de Pearson précisément parce qu’elle demande seulement si y évolue de façon cohérente avec x , pas si ce mouvement est linéaire, si bien qu’un véritable coude courbe ne peut pas la tromper en lui faisant conclure à l’absence de tendance. Theil (Theil, 1950) et Sen (Sen, 1968), développés à vingt ans d’écart et combinés depuis en un seul estimateur, fournissent l’estimation de pente derrière le contraste de Theil-Sen : la médiane de toutes les pentes par paire, une seule valeur aberrante extrême parmi de nombreux points ordinaires ne peut pas déplacer une médiane comme elle déplacerait un ajustement par moindres carrés ordinaires. Le bootstrap d’Efron (Efron, 1979) est le principe de rééchantillonnage derrière le test de stabilité de la section 16, choisi plutôt qu’un intervalle de confiance asymptotique en forme close précisément parce que le coude candidat de ce pipeline a déjà survécu à plusieurs étapes de sélection non linéaires et non lisses au moment où une affirmation de confiance est nécessaire, exactement le cadre où la normalité asymptotique est la plus difficile à justifier et le rééchantillonnage le plus facile.

Méthodologie. López de Prado (López de Prado, 2018) défend longuement, à partir de séries temporelles financières plutôt que d’ajustement de courbes, des blocs de validation croisée contigus plutôt que mélangés dès lors que des observations voisines sont corrélées, puisqu’une répartition mélangée laisse fuir dans l’estimation hors échantillon d’un point l’information de ses propres voisins. Le même argument se transpose directement à l’axe des x d’une courbe : des points voisins sur une

courbe lisse quelconque se ressemblent pour la même raison structurelle que des jours voisins d’une série de prix, ce pourquoi la validation croisée par blocs d’**elbow-helper** (section 16) met de côté des blocs contigus de x plutôt que des points tirés au hasard.

Pour aller plus loin. Hastie, Tibshirani et Friedman ([Hastie et al., 2009](#)), Bishop ([Bishop, 2006](#)), Bouveyron, Celeux, Murphy et Raftery ([Bouveyron et al., 2021](#)), Cormen, Leiserson, Rivest et Stein ([Cormen et al., 2022](#)) et Dasgupta, Papadimitriou et Vazirani ([Dasgupta et al., 2006](#)) sont cités tout au long de cette note comme traitements pédagogiques accessibles, non comme l’origine d’une affirmation particulière ci-dessus ; quelques-uns, tirés de la [liste de livres](#) personnellement tenue par l’auteur, ne sont indiqués que là où directement pertinents.

Deuxième partie

Un coude, la géométrie d'une pliure

3 Mettre les données sur un pied d'égalité : la normalisation

Intuition. Une courbe de fréquentation d'un site web sur plusieurs mois et une courbe de cours boursiers sur plusieurs années n'ont numériquement rien de comparable : l'une peut atteindre les milliers, l'autre rester à quelques unités, et pourtant les deux pourraient partager la même *forme*. Avant de comparer des formes, on ramène les deux axes dans la même boîte, le carré unité $[0, 1] \times [0, 1]$, afin que chaque seuil utilisé plus loin dans le pipeline, ce qui compte pour un virage net, ce qui compte pour un bruit excessif, garde le même sens quelles que soient les unités d'origine des données.

Formule. x est mis à l'échelle linéairement :

$$x_{\text{norm}} = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \quad (8)$$

y est mis à l'échelle de la même façon, mais en utilisant les 5e et 95e percentiles plutôt que le minimum et le maximum bruts, puis écrêté à $[0, 1]$:

$$y_{\text{scaled}} = \text{clip}\left(\frac{y - y_{p5}}{y_{p95} - y_{p5}}, 0, 1\right) \quad (9)$$

Ce choix de percentiles n'a rien de cosmétique : une seule valeur aberrante extrême, une erreur de saisie, un pic isolé, étirerait sinon tout l'axe y pour l'accueillir et écraserait le vrai signal dans une mince bande proche de zéro. Les percentiles absorbent quelques points extrêmes sans déformer la forme qui compte (`src/elbow_helper/preprocessing.py`).

4 La bosse de la courbe de différence : ce que « coude » veut dire, précisément

Intuition. Un coude d'école, en forme de $\sqrt{x}$, monte vite puis se stabilise. Si l'on soustrait de la courbe elle-même la droite diagonale qui relie ses deux extrémités, le résultat est une bosse : elle part de zéro, culmine quelque part au milieu, puis revient à zéro. Le sommet de cette bosse se trouve exactement là où la courbe s'écarte le plus d'une droite, ce qui correspond à l'idée intuitive de « coude ». Cette image se transforme en algorithme ([Satopaa et al., 2011](#)) : soustraire la diagonale, repérer le sommet de la bosse, l'appeler le coude. `locator.py`, dans `elbow-helper`, en est un portage NumPy réécrit de zéro ; voir la section Remerciements du `README.md` du projet pour l'origine de cette implémentation.

Quatre formes existent en principe, concave croissante, concave décroissante, convexe croissante, convexe décroissante, mais une seule d'entre elles, concave croissante, correspond au cas « évident » de bosse au-dessus de la diagonale. `elbow-helper` ramène d'abord les trois autres à ce cas unique (retournement, effet miroir, ou les deux, selon `curve/direction`), applique la même logique de sommet de bosse, puis inverse la transformation sur le résultat obtenu. Voilà pourquoi `curve` et `direction` importent : ils indiquent à l'algorithme laquelle des quatre images miroir il a sous les yeux.

Formule. Une fois ramenée au repère concave croissant, la courbe de différence s'écrit

$$d(x_i) = y_{\text{norm}}(x_i) - x_{\text{norm},i} \quad (10)$$

Ses maxima locaux sont les coudes candidats. À chaque maximum local j , un seuil de sensibilité fixe à quel point la courbe doit redescendre sous le sommet avant que celui-ci soit considéré comme un véritable coude plutôt qu’une simple oscillation :

$$T_j = d_{\text{max},j} - S \cdot |\Delta x_{\text{norm}}| \quad (11)$$

où S est le paramètre de sensibilité, plus S est grand, plus l’exigence est forte, la barre désignant l’espacement moyen entre x consécutifs. En parcourant la courbe de différence de gauche à droite, un coude est déclaré au dernier indice avant que $d(x)$ ne passe sous le seuil T_j en vigueur ; en mode « en ligne », le parcours se réarme à chaque nouveau sommet plus élevé, si bien que seule la dernière bosse, la plus persistante, l’emporte (`src/elbow_helper/locator.py, find_knee`).

5 Le modèle en ligne brisée : un coude comme changement de pente

Intuition. Un véritable coude devrait permettre à une droite en deux morceaux, une pente, puis une pente différente après le coude, de mieux s’ajuster aux données qu’une seule droite. `elbow-helper` construit ce modèle en deux morceaux comme une seule courbe continue, à l’aide d’une fonction charnière, $\max(0, x - k)$, qui vaut exactement zéro avant le coude k et croît linéairement ensuite, de sorte que l’ajouter à une droite simple donne une unique formule pour une droite qui se plie en k sans discontinuité :

$$y = a + b x + c \cdot \max(0, x - k) \quad (12)$$

C’est l’objet algébrique autour duquel s’articule, en définitive, chaque vérification de confiance de la partie IV : l’ajuster par moindres carrés ordinaires et comparer son erreur résiduelle à celle d’une droite simple répond à la question « la pliure aide-t-elle réellement ».

6 Un exemple travaillé : le coude d’inertie du k-means

La docstring de `robust_elbow` désigne elle-même ce cas comme le cas convexe décroissant emblématique. Il vaut la peine de le voir tourner de bout en bout : faire tourner le k-means pour $k = 1, 2, \dots$, tracer l’inertie (la somme des carrés des distances intra-groupe) en fonction de k , et chercher le point au-delà duquel ajouter un groupe de plus cesse d’en valoir la peine.

`tests/test_real_world_examples.py::kmeans_inertia_curve` construit huit amas 2D bien séparés, chacun sa propre ellipse légèrement allongée plutôt qu’une gaussienne isotrope ordinaire, et fait tourner l’algorithme de Lloyd (quinze redémarrages aléatoires par valeur de k , pour éviter qu’un mauvais optimum local ne brouille la courbe) pour $k = 1, \dots, 24$. L’inertie chute de deux ordres de grandeur à $k = 8$, le vrai nombre de groupes. Ce qui se passe après $k = 8$ mérite qu’on s’y attarde : l’inertie continue de décroître, doucement et de façon légèrement irrégulière, plutôt que de se figer d’un coup. Cette pente résiduelle est réelle, pas un artefact de tracé : l’allongement propre à chaque amas laisse, pour un k au-delà de 8, une véritable sous-structure à exploiter, exactement comme les groupes d’un vrai jeu de données ne sont jamais des cercles parfaits. La figure ci-dessous trace l’inertie sur un axe logarithmique pour cette raison précise : la chute à $k = 8$ est tellement plus grande que la lente décroissance de la traîne qu’un axe linéaire aplatirait cette dernière à l’œil, alors même que les valeurs continuent bel et bien de baisser.

Lire la légende de preuves, avant sa propre dérivation. La figure ci-dessous est la première de cette note avec la légende de diagnostic complète qu'`elbow_helper.plotting` dessine à côté de chaque coude confirmé, cinq nombres que les parties III et IV dérivent proprement plus loin ; en langage clair par avance : la *probabilité de détection* (section 17) est la fréquence à laquelle le même coude réapparaît sous une reprise par bootstrap des résidus : proche de 100% signifie que le coude n'est pas un artefact de ce seul tirage de bruit. Le p (*nul*) (valeur p du test nul, section 18) est le degré de surprise qu'aurait le coude observé si la vraie courbe était une droite : petit signifie qu'une explication par une simple droite est peu plausible. Le *contraste de pente* (section 15) est un score $[0, 1]$ pour la netteté du changement de pente au coude : proche de 100% signifie que les deux côtés pointent dans des directions essentiellement opposées ou très différentes. La *probabilité a posteriori* (section 16) est la cote dérivée du BIC, sous une approximation par facteur de Bayes, qu'une ligne brisée s'ajuste réellement mieux qu'une simple droite plutôt que par hasard. La *qualité d'ajustement* (section 1, déjà dérivée) est la proximité de l'ajustement en ligne brisée confirmé aux données, relative à un pire cas délibérément pessimiste : 100% à l'ajustement parfait. Chacun de ces cinq nombres est un score $[0, 1]$ borné et justifié, pas une statistique brute et dépendante d'une échelle qu'il faudrait calibrer à l'œil ; quatre se lisent comme une preuve forte proche de 100%, seul le p nul va dans l'autre sens, petit plutôt que grand, exactement la convention habituelle d'une valeur p .

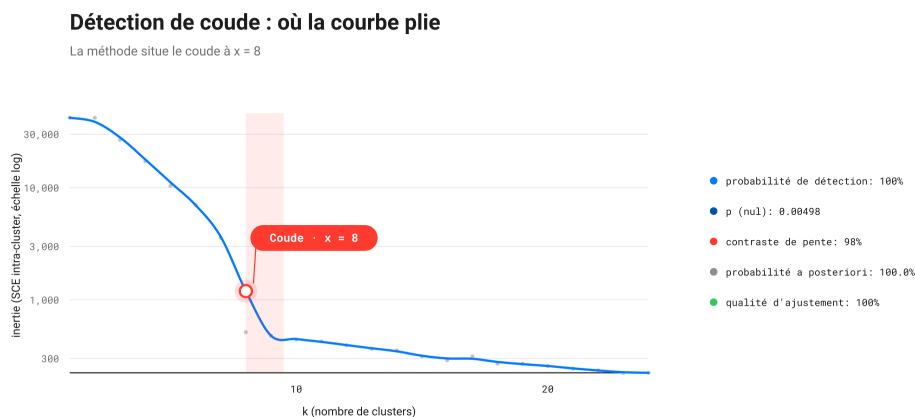


FIGURE 1 – La courbe d’inertie du k-means, axe y en échelle logarithmique, avec le coude détecté par `elbow_helper.robust_elbow` (ligne rouge pointillée) à $k=8$, le vrai nombre de groupes, et une traîne qui continue de décroître doucement au-delà.

Deux pièges pratiques que cet exemple a révélés, et qu’il vaut la peine d’énoncer clairement. D’abord : une courbe d’inertie du k-means n’a qu’un point par valeur de k candidate, rarement plus d’une vingtaine. Son coude authentique se situe en général près du bord *gauche*, à un k petit, pas au milieu. Les valeurs par défaut d’`elbow-helper` (sections 13–18) sont calibrées pour des courbes de mesure plus longues, plus bruitées, dont le coude peut se trouver n’importe où : `min_samples=20` et `min_side_points=5` rejettent d’emblée une courbe aussi courte et déséquilibrée vers la gauche. `tests/test_real_world_examples.py` embarque un profil de `RobustKneeConfig` avec ces deux seuils assouplis, ainsi que plusieurs des seuils de persistance et de bootstrap relâchés en conséquence, et explique dans un commentaire pourquoi exactement : les courbes appliquées courtes et propres relèvent d’un régime authentiquement différent des longues courbes bruitées de type série temporelle que ciblent les valeurs par défaut, ce n’est un défaut ni de l’un ni de l’autre.

Ensuite : la validation croisée par blocs (section 16) découpe une courbe aussi courte, une vingtaine de points, en `cv_folds` morceaux contigus. Les 5 blocs par défaut laissent si peu de points par bloc qu’une frontière de bloc tombant près du coude peut faire basculer le signe de l’amélioration mesurée hors échantillon sur un tirage malchanceux. Trois blocs plus larges au lieu de cinq règlent le problème, une illustration directe et vérifiable du propre argument de la section 16 : la validation croisée a besoin d’assez de points par bloc pour dire quoi que ce soit de stable.

Troisième partie

Plusieurs coudes, la géométrie de plusieurs pliures

7 Le modèle : des segments linéaires indépendants

Intuition. Découper la courbe en morceaux à k points de rupture et ajuster à chacun sa propre droite, indépendamment, en autorisant un saut visible à chaque coupure plutôt que de forcer les morceaux à se rejoindre exactement. C’est un écart délibéré par rapport au modèle à charnière continue de la partie II : dès qu’on laisse libre plus d’un point de rupture, un ajustement linéaire par morceaux *continu* cesse de se décomposer en une somme de coûts indépendants par morceau, déplacer un point de rupture modifie la condition aux limites que chaque morceau voisin doit respecter. Cette dépendance casse les algorithmes de recherche efficaces présentés plus loin. La littérature sur les points de rupture mobilisée et testée dans cette section, Yao, Zhang et Siegmund, PELT, segmentation binaire, est énoncée pour des segments indépendants : adopter ce même modèle est ce qui rend la comparaison loyale envers *leurs* résultats, et non le test d’un modèle différent portant leur nom.

8 Le coût d’un segment, sous forme close

Intuition. Ajuster une droite à travers une poignée de points et mesurer la qualité de l’ajustement, la somme des carrés des résidus (RSS), pourrait sembler exiger de parcourir chaque point, mais un raccourci existe : dès que cinq totaux cumulés sont connus, une somme des x , des y , des x^2 , des xy , des y^2 , le RSS de la droite la mieux ajustée pour n’importe quelle plage de points se calcule en une seule étape, sans retoucher les points un par un. Précalculer ces totaux une fois pour toutes, puis lire instantanément le coût de n’importe quel segment, rend praticable la recherche parmi des milliers de façons de découper la courbe.

Formule. Pour un segment couvrant m points de valeurs $x_1, \dots, x_m$ et $y_1, \dots, y_m$, posons

$$S_1 = m, \quad S_x = \sum_i x_i, \quad S_y = \sum_i y_i, \quad S_{xx} = \sum_i x_i^2, \quad S_{xy} = \sum_i x_i y_i, \quad S_{yy} = \sum_i y_i^2 \quad (13)$$

La pente et l’ordonnée à l’origine des moindres carrés pour $y = a + bx$ valent

$$b = \frac{m S_{xy} - S_x S_y}{m S_{xx} - S_x^2}, \quad a = \frac{S_y - b S_x}{m} \quad (14)$$

et, en substituant dans $RSS = \sum_i (y_i - a - bx_i)^2$ puis en simplifiant à l’aide des équations normales,

$$RSS = S_{yy} - a S_y - b S_{xy} \quad (15)$$

Précalculer une fois les six sommes cumulées coûte $O(n)$; chaque lecture ultérieure du coût d'un segment ne coûte plus que $O(1)$. C'est une pratique courante des logiciels de détection de points de rupture, conséquence algébrique directe des équations normales des moindres carrés, qui ne nécessite pas de citation propre.

9 Le partitionnement optimal : la programmation dynamique

Intuition. Chercher la meilleure façon de placer k points de rupture parmi n points en essayant toutes les combinaisons possibles serait d'une lenteur astronomique. La programmation dynamique évite cela en construisant la réponse à partir de sous-réponses plus petites : la façon la moins coûteuse d'expliquer les t premiers points avec k segments est, pour chaque emplacement possible de la dernière coupure s , la façon la moins coûteuse d'expliquer les s premiers points avec $k - 1$ segments, additionnée du coût d'un dernier segment de s à t , le tout minimisé sur tous les s valides. Comme la meilleure réponse à k segments ne dépend que des meilleures réponses à $(k - 1)$ segments, déjà calculées une fois chacune, aucune combinaison n'est jamais recalculée : c'est le principe d'optimalité de Bellman.

Formule. Avec $C[k][t]$ le RSS total minimal des t premiers points découpés en $k + 1$ segments :

$$C[0][t] = \text{cost}(0, t) \quad (16)$$

$$C[k][t] = \min_s \left(C[k - 1][s] + \text{cost}(s, t) \right) \quad (17)$$

où s parcourt les coupures qui laissent chaque segment d'au moins `min_seg` points. Résoudre cela pour chaque k jusqu'à $k_{\max}$ et chaque t jusqu'à n coûte $O(k_{\max} \cdot n^2)$, chaque cellule étant une minimisation en $O(n)$ sur des coûts de segment en $O(1)$ issus de la section précédente. Des pointeurs arrière permettent de retrouver les positions de coupure effectives. `research/multiknee/tests/test_segmentation.py::` compare ce résultat à l'énumération littérale de chaque partition valide sur de petites entrées : sa correction est donc vérifiée empiriquement, non simplement affirmée à partir de la récurrence du manuel (Cormen et al., 2022; Dasgupta et al., 2006).

C'est la même récurrence qui sous-tend Yao (1988), Zhang et Siegmund (2007), ainsi que PELT, « Pruned Exact Linear Time », dû à Killick, Fearnhead et Eckley (2012). PELT reprend cette récurrence en y ajoutant une étape d'élagage dont on démontre qu'elle ne supprime jamais l'optimum pour une pénalité additive : elle renvoie donc des réponses identiques, seulement plus vite. Aucune implémentation séparée de PELT n'a été construite pour cette raison, puisqu'elle reproduirait exactement ces mêmes chiffres.

10 La segmentation binaire gloutonne : l'alternative plus rapide, moins bonne

Intuition. Plutôt que de résoudre le problème dans son ensemble, faire à répétition le seul meilleur choix immédiatement disponible : trouver la coupure unique qui réduit le plus l'erreur *globale*, s'y engager, puis répéter l'opération sur les deux morceaux obtenus. C'est bien plus rapide que la programmation dynamique, mais le résultat ne peut jamais être meilleur, seulement pire ou égal, puisque les décisions prises tôt se figent avant que des preuves ultérieures existent pour les corriger.

Un échec concret, pas hypothétique. `research/multiknee/tests/test_segmentation.py::test_greedy_can_strictly[...]noiseless` présente une courbe sans bruit avec deux points de rupture nets et réels, où la première coupure gloutonne ne tombe sur aucun des deux ; au moment où l’approche gloutonne ajoute une deuxième coupure, son erreur demeure strictement supérieure à l’optimum exact, à erreur nulle, de la programmation dynamique. `research/multiknee/compare.py` mesure directement la conséquence en aval, sur des courbes linéaires par morceaux synthétiques ($n = 100$, 25 répliques par cellule vrai- k /bruit) : toute méthode fondée sur la programmation dynamique bat son équivalent glouton, DP+mBIC_sub atteignant 0,85 d’exactitude sur k contre 0,60 pour Greedy+mBIC_sub, DP+BIC atteignant 0,65 contre 0,49 pour Greedy+BIC. Les erreurs d’engagement précoce de l’approche gloutonne la poussent en particulier à *surestimer* le nombre de points de rupture à faible bruit, puisque corriger plus tard sa propre erreur de placement ressemble, aux yeux d’un critère de sélection de modèle, à une véritable structure supplémentaire méritant un point de rupture de plus. C’est la motivation classique de la Wild Binary Segmentation ([Fryzlewicz, 2014](#)).

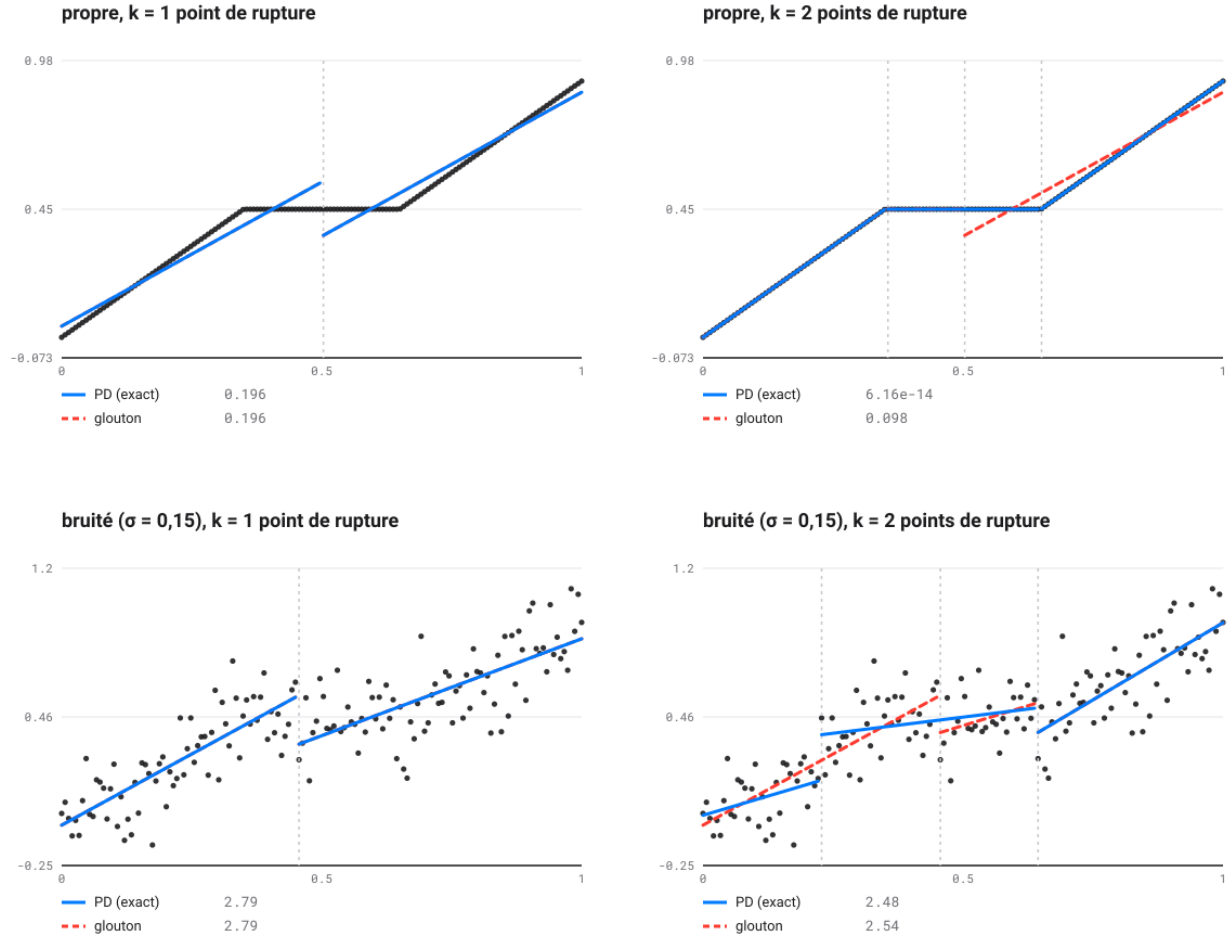
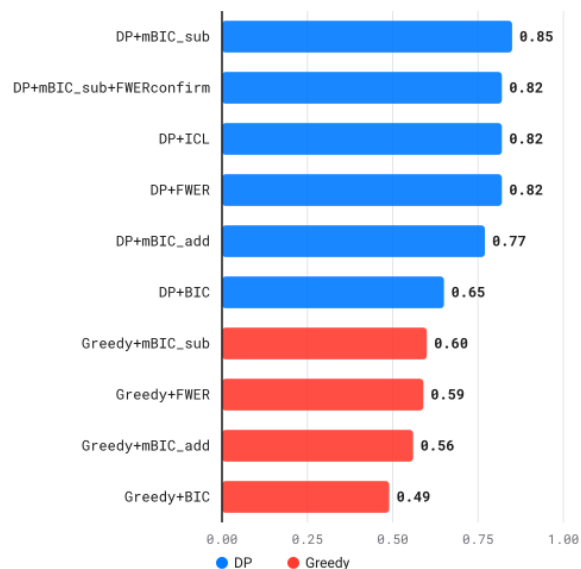


FIGURE 2 – La programmation dynamique (bleu plein) contre l’approche gloutonne (rouge pointillé) sur la même courbe, à $k = 1$ et $k = 2$ points de rupture, propre (en haut) et avec un bruit $\sigma = 0,15$ ajouté (en bas). À $k = 1$, les deux coïncident exactement : une seule coupure n’a rien à engager trop tôt. À $k = 2$, sans bruit, la programmation dynamique retrouve les vrais points de rupture avec une erreur quasi nulle, tandis que la première coupure gloutonne, fixée à l’emplacement de $k = 1$, l’empêche d’atteindre le même ajustement à n’importe quelle étape ultérieure : l’échec concret que le texte ci-dessus décrit. Le bruit adoucit l’écart sans le refermer.

Exactitude sur k par méthode

$P(K = \text{vrai } k), 25 \text{ répliques} \times 4 \text{ vrais } k \times 3 \text{ niveaux de bruit}$



Faux positifs sur données plates

$P(K > 0 \mid \text{vrai } k = 0)$

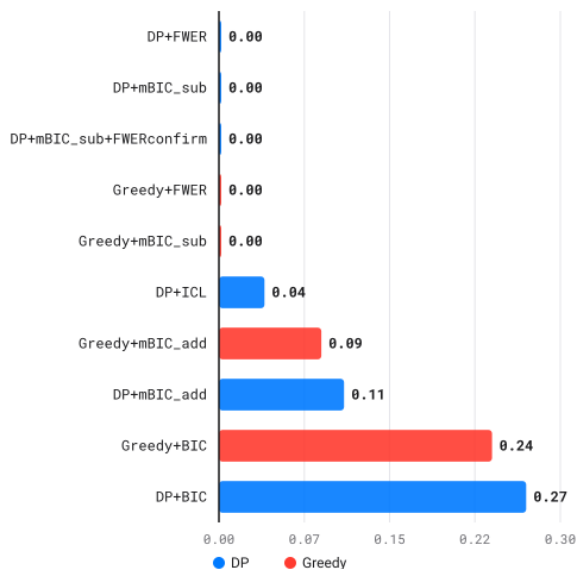


FIGURE 3 – À gauche : exactitude sur k , classée, pour chaque combinaison programmation dynamique (bleu) et gloutonne (rouge) avec les quatre critères à plusieurs coudes que cette note dérive, BIC, mBIC, ICL et le test séquentiel FWER, mesurée sur le même banc d’essai synthétique. Toute méthode fondée sur la programmation dynamique dépasse son équivalent glouton. À droite : le taux de faux positifs de ces mêmes dix méthodes sur des données réellement plates, le BIC nu étant l’exception nette des deux côtés, quelle que soit la stratégie de recherche.

11 Un exemple travaillé : une montagne aux pentes alternées

Le `robust_knee` de la partie II a besoin de `curve/direction` pour savoir laquelle des quatre images miroir il examine, ou doit l’inférer de la forme globale de toute la courbe. Une courbe qui monte, se stabilise, redescend, puis se stabilise à nouveau, une montagne, n’a justement aucune forme globale unique : la géométrie à signe fixe de la section 4 ne peut pas la décrire, quels que soient les réglages de `curve/direction`.

`robust_knees` contourne entièrement la question, puisque le modèle à segments indépendants de cette partie ne suppose jamais un signe commun pour les pentes : chaque segment reçoit sa propre pente libre, si bien qu’un changement de signe d’un segment à l’autre n’est pas un cas particulier, seulement ce que les données montrent.

`tests/test_multiknee.py::alternating_slope_curve` construit exactement cela : montée franche, palier, descente franche, palier, quatre segments et trois points de rupture, chacun un véritable changement de signe ou d’amplitude. `robust_knees` les retrouve tous les trois, dans le bon ordre, avec les bons signes, sur chaque tirage testé.

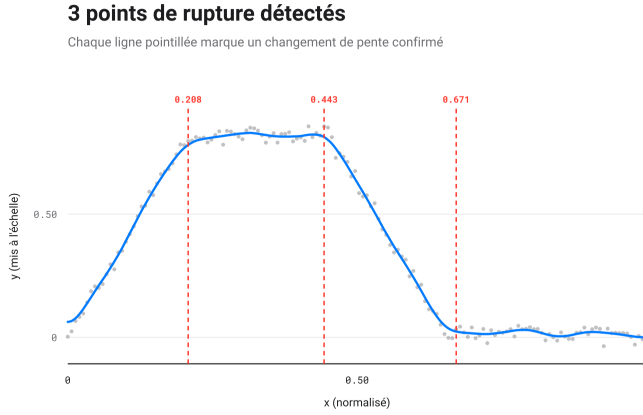


FIGURE 4 – Une courbe en montagne avec les trois points de rupture détectés par `elbow_helper.robust_knees` (lignes rouges pointillées) : montée, palier, descente, palier, retrouvés sans aucun argument `curve` ni `direction`.

Quatrième partie

Un ou plusieurs coudes, avec du bruit

Les parties II et III décrivent la géométrie d’une pliure comme si les données étaient exactes. Les données réelles ne le sont jamais. Rien de ce qui précède n’est donc retenu tel quel. Cette partie est la couche de confiance : une chaîne de vérifications indépendantes qu’un candidat à coude unique doit franchir pour devenir un `ClearKnee`, et un ensemble de critères de sélection de modèle qui décident, pour le cas multi-coudes, combien de points de rupture le bruit permet réellement de soutenir.

12 Y a-t-il seulement une tendance ? La corrélation de rang de Spearman

Intuition. Avant de chercher un coude, `elbow-helper` vérifie une question plus élémentaire : y évolue-t-il seulement de façon cohérente avec x ? Pas « la relation est-elle une droite », la corrélation de Pearson pose cette question-là et se laisserait tromper par une véritable forme de coude courbée, mais « quand x augmente, y a-t-il tendance à monter aussi, ou à descendre, aussi sinueuse soit la courbe ». L’astuce consiste à remplacer chaque valeur par son *rang*, 1er plus petit, 2e plus petit, ainsi de suite, avant de corrélérer. Les rangs éliminent précisément l’information qui rendrait une corrélation ordinaire sensible à la forme exacte de la courbe ou aux valeurs aberrantes, ne conservant que la question « l’ordre est-il respecté ».

Exemple travaillé. Cinq points avec $x = (1, 2, 3, 4, 5)$ et $y = (2, 1, 5, 3, 9)$. Les rangs de y sont $(2, 1, 3, 4, 5)$ (le rang 1 va à la plus petite valeur, donc $y = 1$ en position 2 reçoit le rang 1, et ainsi de suite). Même si les valeurs brutes oscillent $(2, 1, 5, \dots)$, les rangs suivent d’assez près ceux de x , $(1, 2, 3, 4, 5)$, ce qui donne une corrélation positive élevée malgré le creux local.

Formule. Avec r_i, s_i les rangs de x_i, y_i (les ex æquo reçoivent le rang moyen de leur groupe) :

$$\rho = \frac{\sum_i (r_i - \bar{r})(s_i - \bar{s})}{\sqrt{\sum_i (r_i - \bar{r})^2} \sqrt{\sum_i (s_i - \bar{s})^2}} \quad (18)$$

`elbow_helper.numerics.spearman` implémente cela directement, sans dépendance à `scipy.stats.rankdata`. Un coude candidat n'est retenu, à ce stade, que si $|\rho|$ franchit `config.min_spearman_abs` (0,60 par défaut) et qu'une vérification pondérée par l'amplitude, celle du « quelle part du mouvement contredit la direction annoncée », passe elle aussi ; voir `INCOMPATIBLE_GLOBAL_SHAPE` dans la liste des raisons d'abstention du README ([Spearman, 1904](#)).

13 Ne pas se fier à une seule échelle : la recherche lissage $\times$ sensibilité

Intuition. À sa résolution native, une courbe bruitée fait ressembler chaque petite oscillation à un coude candidat. Lissée fortement, elle peut au contraire faire disparaître un véritable coude dans le flou. Aucun des deux extrêmes n'est fiable pris isolément, si bien qu'`elbow-helper` fait tourner le localisateur sur toute une grille : plusieurs largeurs de lissage gaussien, de 1, aucun lissage, jusqu'à environ un quart des données, croisées avec plusieurs sensibilités S . Chaque exécution qui trouve un coude verse un candidat dans un réservoir commun ; rien n'est encore retenu à ce stade, on ne fait que *proposer*.

14 Ne le compter que s'il survit partout : le regroupement par persistance

Intuition. Un coude authentique devrait apparaître à presque toutes les largeurs de lissage et toutes les sensibilités, en ne se déplaçant que légèrement. Un coude qui n'est en réalité que du bruit a tendance à sauter de façon imprévisible quand le lissage change, ou à n'apparaître qu'à un ou deux réglages avant de disparaître ailleurs. `elbow-helper` regroupe l'ensemble des candidats par emplacement, à `cluster_tolerance` près, en unités x normalisées, puis pose au plus grand groupe les questions suivantes : s'étend-il sur plusieurs échelles de lissage *consécutives*, apparaît-il à la plupart des sensibilités, sa dispersion (écart absolu médian) est-elle resserrée ? Si deux groupes sont à la fois grands et comparablement soutenus, le pipeline refuse explicitement de désigner un gagnant (`MULTIPLE_PLAUSIBLE_KNEES`) plutôt que de deviner (`src/elbow_helper/clustering.py`).

15 Confirmer la pente : le contraste de Theil-Sen

Intuition. La façon ordinaire d'estimer une pente, ajuster une droite à un ensemble de points, se laisse facilement perturber par un seul point aberrant éloigné des autres. L'estimateur de Theil-Sen contourne le problème : calculer la pente entre *chaque paire* de points, puis prendre la médiane de toutes ces pentes. Une poignée de paires abîmées, impliquant une valeur aberrante, se retrouvent minoritaires face à la majorité des bonnes paires.

Exemple travaillé. Trois points $(0, 0), (1, 1), (2, 100)$, le dernier étant une valeur aberrante extrême. Pentes par paires : $(1 - 0)/(1 - 0) = 1$, $(100 - 0)/(2 - 0) = 50$, $(100 - 1)/(2 - 1) = 99$. La médiane de $\{1, 50, 99\}$ vaut 50, encore tirée par la valeur aberrante avec seulement trois points, mais avec davantage de points normaux et une seule valeur aberrante, la médiane cesse de bouger dès que les valeurs aberrantes deviennent minoritaires, contrairement à un ajustement par moindres carrés ordinaire, que la valeur aberrante dominerait immédiatement.

Formule.

$$\hat{\beta} = \text{median}_{i < j} \frac{y_j - y_i}{x_j - x_i} \quad (19)$$

`elbow-helper` calcule cela sur une petite fenêtre juste avant puis juste après le coude candidat, puis compare les deux pentes :

$$\text{contrast} = \frac{|m_{\text{left}} - m_{\text{right}}|}{|m_{\text{left}}| + |m_{\text{right}}| + \epsilon} \quad (20)$$

Un contraste inférieur à `config.min_slope_contrast` (0,30 par défaut) fait échouer cette vérification (`WEAK_SLOPE_CHANGE`) (Theil, 1950) (Sen, 1968).

16 Confirmer le modèle : le BIC et la validation croisée par blocs

Intuition. Tout paramètre libre supplémentaire dans un modèle ne peut que l’aider à mieux s’ajuster aux données *d’entraînement*, de façon purement mécanique, qu’il capture ou non quelque chose de réel : un modèle avec autant de paramètres que de points de données s’ajuste parfaitement et n’explique rien. Le critère d’information bayésien, correction apportée par Schwarz à ce problème, facture à chaque paramètre supplémentaire un péage fixe, en unités d’amélioration de vraisemblance logarithmique nécessaire pour que cela en vaille la peine, de sorte qu’un modèle n’est préféré que s’il franchit cette barre.

D’où vient la base du logarithme. La sous-section 1 dérive, à partir de la vraisemblance gaussienne brute $L(\theta, \sigma^2)$, que la log-vraisemblance négative

$$\text{NLL} = n \ln \left(\frac{RSS}{n} \right) + n(1 + \ln 2\pi) \quad (21)$$

est en logarithme naturel par construction, héritée de l’exp propre à la densité gaussienne, pas choisie librement, et que ses cas extrêmes se lisent mieux via MSE_{pire} , une référence finie délibérément pessimiste, plutôt que via $\text{MSE} \rightarrow \infty$ ou la moyenne d’échantillon, trop facile à battre. Toute formule de la famille BIC dans cette note, y compris celle ci-dessous, s’ouvre avec ce même terme $n \ln(RSS/n)$.

Formule. La correction de Schwarz au problème de surapprentissage du paragraphe Intuition ajoute une pénalité de complexité, $p \ln n$, par-dessus ce même NLL, via une approximation de Laplace de la vraisemblance marginale du modèle, elle aussi menée en logarithme naturel de bout en bout. En laissant tomber la constante additive $n(1 + \ln 2\pi)$, qui s’annule quand on compare deux modèles sur les mêmes données :

$$\text{BIC} = n \ln \left(\frac{RSS}{n} \right) + p \ln n \quad (22)$$

Plus c’est bas, mieux c’est. `elbow_helper.numerics.bic` implémente exactement cela ; le pipeline à coude unique compare le BIC de la droite simple ($p = 2 + 1$) à celui de la ligne brisée ($p = 3 + 1$) de la section 3, en exigeant une amélioration d’au moins `config.min_bic_improvement` (10 par défaut) (Schwarz, 1978) (Hastie et al., 2009) (Bishop, 2006). Toute quantité en aval hérite de cette même base naturelle par construction : le $\text{BIC}(k)$ à plusieurs coudes de la section 19, le mBIC et l’ICL sont chacun construits en ajoutant d’autres termes de pénalité à ce même NLL, jamais en redérivant la vraisemblance depuis zéro. Changer cette base n’importe où dans cette chaîne reviendrait à redériver la densité gaussienne elle-même dans cette base, pas à simplement redimensionner le nombre final.

Le BIC vérifie seulement si le paramètre supplémentaire vaut sa pénalité *sur les données ayant servi à l’ajuster*. La validation croisée par blocs pose une question complémentaire, plus méfiante : la ligne brisée l’emporte-t-elle encore une fois testée sur des données qu’elle n’a jamais vues pendant

l'ajustement ? Une validation croisée ordinaire, mélangée, laisserait fuir de l'information ici, puisque des points voisins sur une courbe se ressemblent ; `elbow-helper` met donc de côté, tour à tour, des blocs *contigus* de x , et non des points tirés au hasard, ce qui imite l'allure qu'aurait la courbe si un segment vraiment inédit venait à manquer. C'est la même exigence de rigueur que défend longuement le livre de Marcos López de Prado sur le machine learning financier, dans le contexte de données ordonnées et séquentielles (2018).

17 Le bootstrap : le coude survit-il à une reprise ?

Intuition. Si la collecte des données était refaite, le même coude réapparaîtrait-il ? Ou le coude observé n'était-il qu'un coup de chance lié au bruit particulier de cette exécution ? Comme une véritable reprise n'est pas disponible, le bootstrap en simule une : prendre les résidus laissés par l'ajustement du modèle en ligne brisée accepté, la part de y que le modèle n'a pas expliquée, les rééchantillonner avec remise, les rajouter au modèle ajusté pour construire une courbe de « reprise » synthétique, puis relancer la recherche *entière* sur celle-ci. Répéter l'opération de nombreuses fois, le bootstrap d'Efron. Un coude qui n'apparaît que dans l'exécution d'origine, et rarement dans les reprises rééchantillonnées, était probablement un coup de chance.

Formule. Pour chacune des B répétitions, $y^* = \hat{y} + r^*$ où r^* est un rééchantillonnage avec remise des résidus observés. `elbow-helper` exige que le coude soit redétection dans au moins `config.min_bootstrap_detection` (90 % par défaut) des répétitions, avec un intervalle à 90 % resserré et unimodal, l'écart entre les 5e et 95e percentiles des emplacements redétection (Efron, 1979) (Hastie et al., 2009).

18 Le test nul : une droite pourrait-elle expliquer cela par hasard ?

Intuition. La dernière vérification, la plus méfiante : simuler de nombreuses courbes qui n'ont *aucun* véritable coude, des droites portant un bruit à la même échelle estimée que les résidus du modèle accepté, exécuter exactement la même procédure de recherche et de confirmation sur chacune, et compter à quelle fréquence cette procédure parvient tout de même à rapporter un coude aussi fort que celui réellement observé. Si le pur bruit produit régulièrement quelque chose d'aussi fort, le coude observé n'est pas une preuve fiable d'une véritable pliure : c'est simplement à quoi ressemblent, parfois, des droites bruitées.

Formule. Avec $B = \text{config.null_replicates}$ (200 par défaut) répétitions de Monte-Carlo et une statistique de test ajustée à la recherche, lexicographique, de sorte qu'une répétition nulle ne « batte » le coude observé que si elle franchit *les mêmes* barrières de confirmation, et pas seulement un score brut :

$$p = \frac{1 + \#\{\text{répétitions nulles au moins aussi fortes que l'observation}\}}{B + 1} \quad (23)$$

Le +1 au numérateur et au dénominateur est la correction usuelle à taille finie qui empêche une valeur p de Monte-Carlo d'annoncer un jour un zéro exact. `elbow-helper` exige $p \leq \text{config.max_null_p_value}$ (0,01 par défaut) (`src/elbow_helper/null_test.py`, dont la docstring du module énonce cette formule exacte).

Seul un candidat qui franchit chacune des vérifications des sections précédentes devient un `ClearKnee`.

19 Le BIC nu pour plusieurs coudes : le critère naïf et pourquoi il surestime

Intuition. Le BIC de la section 16 pénalise le nombre de paramètres de *régression* libres, mais choisir où placer k points de rupture parmi environ n positions possibles est lui-même une forme de liberté, que le BIC nu ne facture jamais. C’est comme si un examen à choix multiples ne retirait des points que pour les mauvaises réponses aux questions tentées, en ignorant combien de questions il y avait à choisir au départ.

Formule. Avec $p = 2(k + 1) + 1$, deux paramètres de régression par segment indépendant, plus la variance du bruit :

$$\text{BIC}(k) = n \ln \left(\frac{\text{RSS}(k)}{n} \right) + p \ln n \quad (24)$$

La conséquence n’est pas que théorique : la figure 3 mesure un taux de faux positifs de 27 % pour ce critère exact (DP+BIC) sur des données sans point de rupture réel, et seulement 0,65 de précision globale exacte en k , le pire des quatre critères fondés sur la programmation dynamique testés (Yao, 1988) (Zhang and Siegmund, 2007).

La raison plus profonde de cette sous-facturation : l’emplacement d’un point de rupture n’est pas un paramètre « régulier » au sens technique que suppose la dérivation du BIC. La log-vraisemblance n’est pas lisse, deux fois différentiable, en l’emplacement, si bien que l’estimateur de l’emplacement ne converge pas au taux habituel $\sqrt{n}$ avec une limite gaussienne : il converge plus vite, au taux n , avec une distribution limite donnée par l’argmax d’une marche aléatoire. Facturer $\frac{1}{2} \ln n$ par paramètre, l’argument d’approximation de Laplace derrière le BIC, suppose exactement la régularité qui s’effondre ici.

20 Le BIC modifié : la correction de Zhang et Siegmund et une convention de signe résolue par le test

Intuition. Si le BIC nu sous-facture la liberté de placer des points de rupture, la correction consiste à facturer davantage, en particulier *davantage pour des segments très courts ou très inégaux*, puisqu’un segment minuscule offre bien plus de liberté de placement, de nombreuses positions voisines paraissant presque aussi bonnes, qu’un segment couvrant une large portion bien définie des données.

Formule. La pénalité de Zhang et Siegmund, rapportée ici via une source secondaire plutôt que l’article original, s’écrit

$$\text{penalty}(k) = 3k \ln n + \sum_{j=1}^{k+1} \ln \left(\frac{\ell_j}{n} \right) \quad (25)$$

où ℓ_j sont les longueurs de segments obtenues. Le 3, au lieu du 2 implicite du BIC nu issu des deux seuls paramètres de régression, constitue déjà à lui seul une charge de complexité plus forte ; le second terme est toujours ≤ 0 et, par la concavité du logarithme, se rapproche de zéro quand les segments sont équilibrés, et s’en éloigne le plus quand ils sont très inégaux.

Une ambiguïté réelle, résolue empiriquement plutôt que supposée. En intégrant cette pénalité par addition directe dans un critère où plus c’est bas, mieux c’est,

$$\text{mBIC}_{\text{additive}}(k) = n \ln \left(\frac{RSS(k)}{n} \right) + 3k \ln n + \sum_{j=1}^{k+1} \ln \left(\frac{\ell_j}{n} \right) \quad (26)$$

le cas le plus négatif, celui des segments inégaux, abaisse le total : cette combinaison *récompense* les segments inégaux, l'inverse du comportement attendu, « pénaliser les segments courts et inégaux ». En soustrayant plutôt le même terme,

$$\text{mBIC}_{\text{subtractive}}(k) = n \ln \left(\frac{RSS(k)}{n} \right) + 3k \ln n - \sum_{j=1}^{k+1} \ln \left(\frac{\ell_j}{n} \right) \quad (27)$$

on pénalise bien les segments inégaux, conformément à l'intention affichée par la littérature. Les deux formes sont implémentées et comparées directement : `modified_bic_subtractive` atteint une précision globale de 0,85 et un taux de faux positifs de 0 % à true $k = 0$; `modified_bic_additive` bat encore le BIC nu (0,77 contre 0,65) mais laisse un taux de faux positifs de 11 %. **C'est la forme soustractive qui est livrée.**

21 L'ICL : des modèles de mélange aux points de rupture, avec un *bug* trouvé et corrigé au passage

Intuition. L'Integrated Completed Likelihood de Biernacki, Celeux et Govaert, conçue pour choisir le nombre de groupes dans un modèle de mélange, ajoute une considération supplémentaire au-dessus du BIC : non seulement « ce modèle s'ajuste-t-il bien compte tenu de sa complexité », mais aussi « les groupes obtenus sont-ils réellement non ambigus ». Deux groupes qui se chevauchent, difficiles à distinguer, sont pénalisés même s'ils s'ajustent aux données à peu près aussi bien que deux groupes nets et bien séparés. Gilles Celeux, coauteur de l'article original sur l'ICL, a plus tard coécrit un ouvrage entier consacré à cette même famille de méthodes (Bouveyron et al., 2021).

Pour les points de rupture, l'équivalent de « à quel groupe appartient ce point » devient « où se situe exactement la coupure » : l'incertitude pertinente porte sur la *segmentation discrète entière*, non sur des étiquettes individuelles associant chaque point à un groupe. Rigai, Lebarbier et Robin fournissent la machinerie nécessaire : une distribution a posteriori exacte, non asymptotique, sur chaque façon de placer $K - 1$ points de rupture, via un programme dynamique avant-arrière structuellement identique à l'algorithme avant-arrière d'un modèle de Markov caché, exécuté sur des segmentations plutôt que sur des états cachés (2012) ; Cleyen, Luong, Rigai et Nuel s'appuient directement sur ce résultat (2013).

Formule. Une passe avant calcule, en espace logarithmique, la masse de vraisemblance totale sur chaque partition valide à K segments des t premiers points, à l'aide de la log-vraisemblance gaussienne du segment

$$\ell(i, j) = -\frac{RSS(i, j)}{2\sigma^2} - \frac{j - i}{2} \ln(2\pi\sigma^2) \quad (28)$$

$$\log Z_1(t) = \ell(0, t), \quad \log Z_K(t) = \log \sum_s \exp \left(\log Z_{K-1}(s) + \ell(s, t) \right) \quad (29)$$

L'entropie de la distribution a posteriori discrète résultante sur les segmentations, $H(K) = \log Z_K(n) - \mathbb{E}[\ell(S)]$, s'estime par échantillonnage arrière de Monte-Carlo, en tirant des segmentations complètes selon la distribution a posteriori puis en moyennant leurs log-probabilités, plutôt que par une forme close.

Un bug que les tests ont attrapé, non un choix de conception. La première version construite ici définissait le score directement comme $-\log Z_K(n) + H(K)$, pure vraisemblance intégrée additionnée d’entropie, sans pénalité de complexité séparée, en supposant que la distribution a posteriori exacte « contenait » déjà toute la notion de complexité nécessaire. Un test sur soixante points de pur bruit a infirmé cette hypothèse : $\log Z_K(n)$ est passé de 89,8 ($K = 1$) à 94,7, 99,0, puis 102,8 nats à mesure que K montait jusqu’à 4, du simple fait de sommer sur un nombre combinatoirement croissant de segmentations candidates, tandis que l’entropie ne montait que de 0 à 3,7, 5,9, puis 7,9 nats sur les mêmes étapes, trop lentement pour compenser cette croissance. Le score obtenu continuait donc de s’améliorer à mesure que K augmentait, même sur du pur bruit, exactement l’échec par surestimation que cet exercice tout entier vise à éviter.

La correction découle d’une relecture de la forme du modèle de mélange ci-dessus : l’ICL est *le BIC augmenté* d’une correction d’entropie, et non un remplacement de la propre pénalité de complexité du BIC :

$$\text{ICL}(k) = \text{BIC}(k) + 2H(K), \quad K = k + 1 \quad (30)$$

Retestée sur les mêmes données de bruit pur, cette version sélectionne correctement $k = 0$; la figure 3 montre qu’une fois corrigée, elle se situe dans la même tranche de performance que le mBIC soustractif (0,82 contre 0,85 de précision globale) (Biernacki et al., 2000).

22 Un test séquentiel à taux d’erreur par famille contrôlé

Intuition. Indépendamment de tout score de la famille du BIC, une question plus directe peut être posée à chaque étape : en parcourant la séquence de segmentations $k = 1, 2, \dots$, la réduction d’erreur due à l’ajout du point de rupture k dépasse-t-elle ce que produirait le pur hasard ? On y répond comme un test de permutation répond à toute question de ce genre : rééchantillonner de nombreuses fois à quoi ressemble le « hasard », puis regarder à quelle fréquence le hasard seul dépasse ce qui a réellement été observé.

Formule. À chaque étape, les résidus $r = y - \hat{y}_{k-1}$ du modèle à $(k - 1)$ points de rupture accepté sont rééchantillonnés avec remise, encore le bootstrap d’Efron, pour construire B courbes synthétiques $y^* = \hat{y}_{k-1} + r^*$. Sur chacune, on calcule la plus grande réduction d’erreur qu’apporterait une coupure de plus, n’importe où, ce qui donne une distribution nulle :

$$p = \frac{1 + \#\{\text{répétitions dont la réduction nulle} \geq \text{réduction observée}\}}{B + 1} \quad (31)$$

Le point de rupture k n’est accepté que si $p \leq \alpha/k_{\max}$, une correction de Bonferroni appliquée sur jusqu’à $k_{\max}$ tests séquentiels. La procédure s’arrête au premier point de rupture qui échoue, si bien qu’elle ne peut jamais « sauter » un point de rupture rejeté pour en accepter un plus tard. Bonferroni est une correction conservatrice pour des tests indépendants ; ces tests séquentiels ne le sont pas, chacun étant conditionné au modèle accepté à l’étape précédente, ce qui maintient le taux d’erreur par famille réel (FWER, family-wise error rate), la probabilité d’accepter au moins un point de rupture parasite n’importe où dans la séquence, au plus égal au α nominal, jamais supérieur.

La figure 3 montre que cette approche atteint un taux de faux positifs de 0 % à true $k = 0$ et une précision globale de 0,82, dans la même tranche que le mBIC soustractif et l’ICL corrigé. Deux alternatives plus rigoureuses ont été envisagées puis écartées : le test asymptotique de Davies (1977) et la garantie multiéchelle de SMUCE, « Simultaneous Multi-scale Change-point Estimator » (Frick et al., 2014), au profit d’un test de permutation dont la correction se vérifie directement par simulation.

23 Ce que cela laisse à l'API livrée

`elbow_helper.robust_knees` livre exactement la combinaison validée ci-dessus : recherche par programmation dynamique (section 9), le BIC modifié à signe soustractif comme critère principal (section 18), et la barrière séquentielle à taux d'erreur par famille contrôlé (section 20) superposée par défaut, en retenant $\min(k \text{ du mBIC}, k \text{ du FWER})$. Ce choix suit la priorité de conception assumée par `elbow-helper` : minimiser les coudes faussement positifs, même au prix de davantage d'abstentions. Le BIC nu, le mBIC additif, l'ICL et la recherche gloutonne restent dans `research/multiknee/` comme alternatives testées puis écartées, ou testées puis jugées trop coûteuses, qui justifient ce choix, sans être livrées comme options de configuration publiques : la surface de l'API publique reflète ainsi ce qui a réellement fait ses preuves.

24 Un exemple travaillé : un escalier de sigmoïdes

Chaque section ci-dessus est soit une formule, soit une affirmation vérifiée par un test. Voici une courbe, passée dans le code réellement livré, pour voir où toute la chaîne aboutit concrètement.

Une sigmoïde logistique, $1/(1 + e^{-\text{steepness}(x - \text{center})})$, monte doucement de 0 à 1 autour de son centre et n'a littéralement aucun point de rupture nulle part : elle est indéfiniment différentiable, sans la moindre cassure à laquelle une recherche linéaire par morceaux pourrait s'accrocher en principe. Sommer trois sigmoïdes à des centres différents, chacune mise à l'échelle pour ajouter une « marche », construit un escalier lisse : trois montées séparées par des paliers plats, la forme qui apparaît chaque fois que plusieurs seuils indépendants sont franchis en séquence, trois cohortes distinctes d'utilisatrices et utilisateurs, chacune montant en puissance d'adoption une semaine différente, ou trois capteurs distincts saturant chacun à une charge différente.

Un modèle linéaire par morceaux, appliqué à une courbe lisse comme celle-ci, ne peut représenter une montée avec un seul segment droit sans tronquer soit un palier plat, soit la montée elle-même. Le compromis auquel s'arrête `robust_knees` est visible directement sur la figure ci-dessous : chacune des trois montées se retrouve encadrée par *deux* points de rupture, l'un où le palier plat se termine et où la montée commence, l'autre où la montée se termine et où le palier suivant commence. Trois montées, six points de rupture, sans exception sur cinq tirages de bruit différents testés (`tests/test_multiknee.py::test_sigmoid_staircase_brackets_each_rise_with_a_breakpoint_pair`).

Un piège pratique que cet exemple a révélé, et qu'il vaut la peine d'énoncer clairement : la correction de Bonferroni de la barrière à taux d'erreur par famille contrôlé divise α par $k_{\max}$, si bien que la plus petite valeur p que le test de permutation puisse produire, $1/(n_{\text{permutations}} + 1)$, doit rester sous $\alpha/k_{\max}$, sans quoi la barrière ne peut jamais passer, aussi réel soit l'effet. Avec le $k_{\max} = 4$ par défaut et `fwer_permutations = 200`, $1/201 \approx 0,005 < 0,05/4 = 0,0125$, largement en dessous. Augmenter $k_{\max}$, comme l'exige cet exemple à six points de rupture, sans relever aussi `fwer_permutations` verrouille silencieusement la barrière : c'est exactement ce qui s'est produit lors de la première tentative de cet exemple ($k_{\max} = 8$, les 100 permutations par défaut, chaque point de rupture rejeté, $k = 0$), repéré en examinant les diagnostics plutôt qu'en supposant que l'algorithme avait échoué pour une raison plus mystérieuse.

25 Un exemple travaillé : le même coude discret, à deux niveaux de bruit

Chaque vérification des sections 13–18 existe pour répondre honnêtement à une seule question : ce coude est-il réel ? Ou est-ce simplement à quoi ressemble le bruit, parfois ? La façon la plus nette

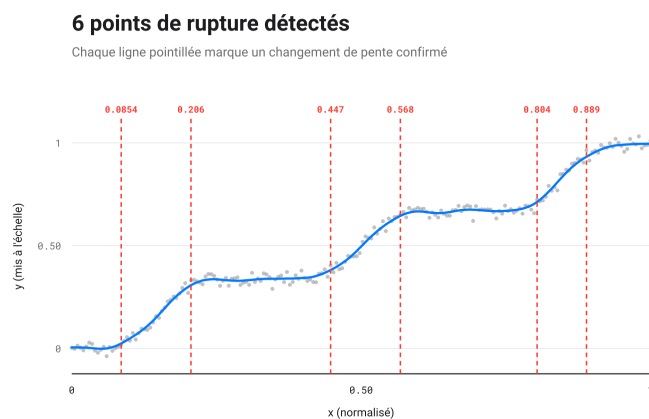


FIGURE 5 – Un escalier lisse construit à partir de trois sigmoïdes logistiques, avec les six points de rupture détectés par `elbow_helper.robust_knees` (lignes rouges pointillées) encadrant chacun le début et la fin d’une montée.

de voir la réponse est de garder la forme réelle fixe et de ne faire varier que le bruit.

`tests/test_multiknee.py::subtle_knee_curve` est délibérément peu spectaculaire : un petit changement de pente bien réel en $x = 0,5$, loin de la netteté de l’escalier de sigmoïdes ou de la montagne. À faible bruit, `robust_knees` le détecte de façon fiable, sur chaque tirage testé. À fort bruit, avec exactement la même forme réelle en dessous, il s’abstient le plus souvent, rapportant zéro point de rupture plutôt que de deviner.

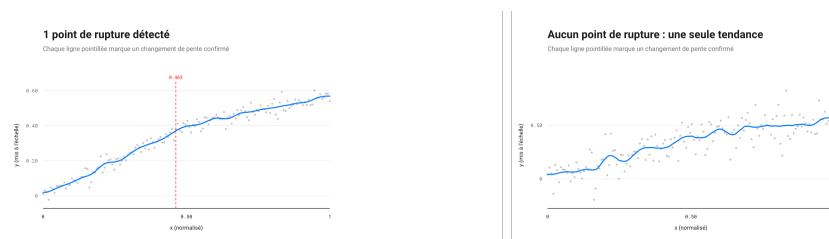


FIGURE 6 – Le même coude discret à deux niveaux de bruit : à faible bruit (gauche), détecté avec un point de rupture ; à fort bruit (droite), abstention avec zéro point de rupture, la même forme réelle sous les deux panneaux.

C’est la priorité de conception de la section 21 rendue visible sur une seule courbe : `elbow-helper` est construit pour se tromper dans le sens honnête. Manquer un coude réel à fort bruit est un coût que cette bibliothèque accepte délibérément ; rapporter un coude qui n’est en réalité que du bruit est précisément l’échec que chaque vérification de la partie IV existe pour écarter.

26 Un exemple travaillé : le diagramme des éboulis de l’ACP

L’autre cas que la docstring de `robust_elbow` désigne directement : un diagramme des éboulis, les valeurs propres d’une matrice de covariance en ordre décroissant, utilisé pour décider combien de composantes principales portent un vrai signal plutôt que du bruit.

`tests/test_real_world_examples.py::pca_scree_curve` construit des données avec six véri-

tables dimensions de signal (grande variance) noyées dans dix-neuf dimensions de bruit, tournées de sorte que le signal ne soit aligné sur aucun axe mesuré directement, exactement la situation que l'ACP est conçue pour démêler. Les dimensions de bruit ne sont pas à variance égale : leur variance décroît de façon géométrique, la forme que prend réellement un spectre de bruit de Marchenko-Pastur, pas le bloc plat qu'une première version de cet exemple produisait avant que cet écart ne soit repéré. La courbe de valeurs propres obtenue chute d'un ordre de grandeur juste à la frontière signal/bruit, puis continue de décroître, lentement et sans à-coups, plutôt que de se stabiliser sur un plateau.

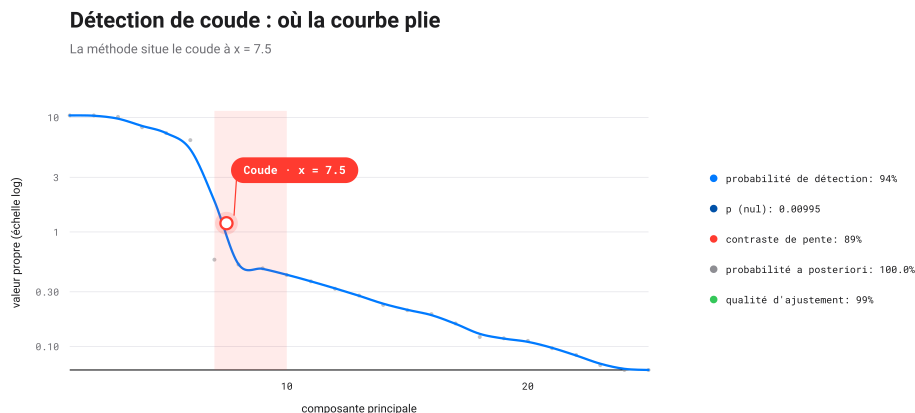


FIGURE 7 – Le diagramme des éboulis de l'ACP, avec le coude détecté par `elbow_helper.robust_elbow` (ligne rouge pointillée) à la frontière entre 6 composantes de signal et 19 composantes de bruit.

Cet exemple partage exactement le piège pratique de la section 4 : un diagramme des éboulis est court et déséquilibré vers la gauche de la même façon qu'une courbe d'inertie, et a donc besoin du même profil assoupli de `RobustKneeConfig`. Il révèle aussi une convention qu'il vaut la peine de connaître avant de lire `knee_x` sur n'importe quelle courbe appliquée courte : la règle du localisateur, « le dernier point avant la pliure », place systématiquement le coude une à deux composantes *après* la dernière véritable composante de signal, pas exactement dessus. Sur chaque tirage testé, le coude détecté tombe à 6 ou 7 composantes pour une dimension de signal réelle de 6, jamais en dessous, un décalage systématique et prévisible plutôt qu'un effet du bruit.

27 Un exemple travaillé : une courbe de saturation exponentielle

`examples/exponential_saturation.py` fait tourner `robust_knee` sur une forme qu'aucun exemple ci-dessus n'a encore couverte : $y = 1 - e^{-t/\tau}$, une tension de charge de circuit RC, une courbe d'apprentissage, une saturation dose-réponse, toute grandeur qui monte vite puis s'aplatit vers un plafond. Contrairement à un vrai coude en ligne brisée, cette courbe n'a de véritable rupture de pente nulle part : elle est lisse et indéfiniment dérivable, un test de résistance délibéré pour un localisateur construit pour trouver des ruptures dans des tendances linéaires par morceaux.

Le piège pratique que révèle cet exemple est un décalage entre deux lectures raisonnables de « où la courbe s'aplatit ». La constante de temps τ des manuels marque $1 - e^{-1} \approx 63\%$ de la montée totale, un repère propre et de forme close. Sur chaque tirage testé, `robust_knee` se place au contraire systématiquement autour de $t/\tau \approx 1.9-2.0$, ce qui correspond à $1 - e^{-2} \approx 85\%$, la même convention

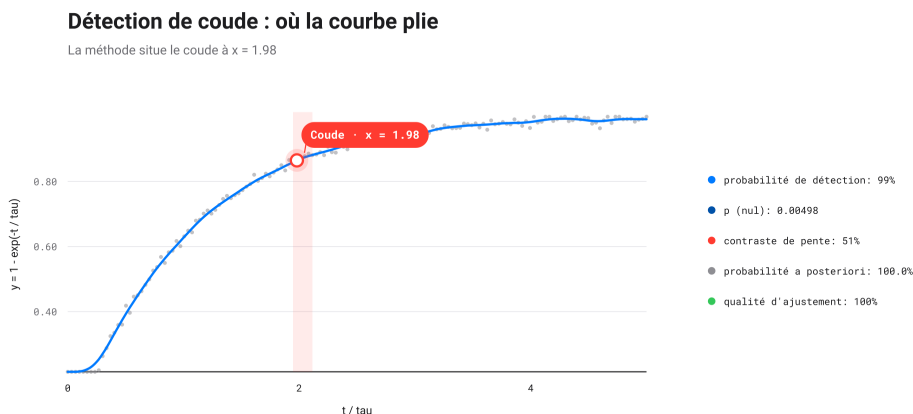


FIGURE 8 – La courbe de charge RC $y = 1 - e^{-t/\tau}$ avec un bruit léger, et le coude détecté par `elbow_helper.robust_knee` (ligne rouge pointillée et bulle) à $t/\tau \approx 1.98$.

« pratiquement stabilisée » utilisée informellement en électronique RC et en automatique. Un œil humain lit la courbe de la même façon : à τ , elle monte encore visiblement, pas encore plate. Ce n'est pas un défaut à corriger : c'est ce que signifie « le dernier point avant la pliure » pour une courbe qui ne cesse jamais vraiment de plier. Le coude localisé dépend aussi de la fenêtre d'observation, ici $t \in [0, 5\tau]$, puisque la décision porte sur l'écart maximal à la corde qui couvre la plage réellement fournie, pas sur une propriété de τ seul.

28 Un exemple travaillé : latence de file d'attente et coude de dimensionnement

`examples/queueing_latency.py` fait tourner `robust_knee` sur une forme qu'aucun exemple ci-dessus ne prend : le temps de réponse d'une file d'attente à un seul serveur croît comme $1/(1 - \rho)$ en fonction de son utilisation ρ , un emballage classique de type M/M/1. La latence reste proche du temps de service de base sur la majeure partie de la plage, puis grimpe brutalement à mesure que ρ approche 1. Cette courbe est convexe et croissante plutôt que les formes concaves et saturantes vues jusqu'ici : le coude ne marque pas un plafond mais une accélération, le point au-delà duquel un peu plus de charge coûte beaucoup plus de latence.

Le coude se situe à $\rho \approx 0,60$, bien avant l'emballement mathématique à $\rho = 1$, pour une latence d'environ 2,5 fois le temps de service de base. Une file d'attente paie déjà un coût réel à 60% d'utilisation : la courbe ne paraît plate jusque-là que parce que $1/(1 - \rho)$ reste petit en échelle absolue. Dimensionner la capacité sur « est-ce que ça a encore l'air à peu près plat » plutôt que sur ce coude localisé est une façon courante et évitable de voir un plan de capacité manquer de marge plus tôt que prévu.

29 Un exemple travaillé : dimensionnement de cache et rendements décroissants

`examples/cache_hit_rate.py` répond avec le même outil à une question familière d'infrastructure : quelle taille donner à un cache ? Pour un jeu de données dont la popularité suit à peu près une

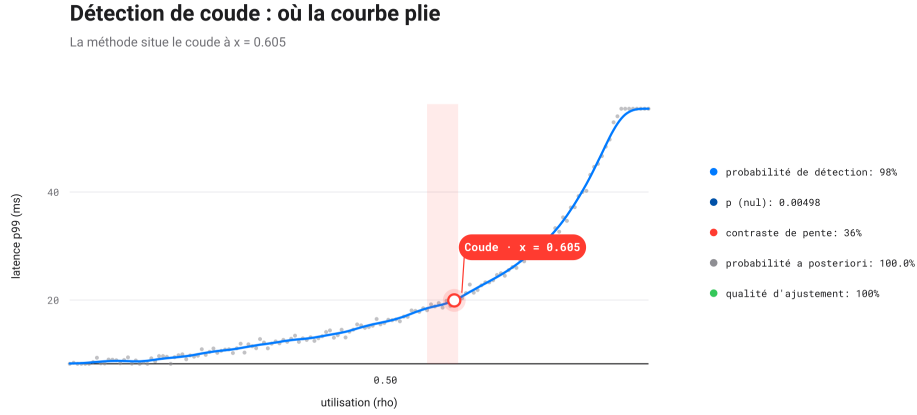


FIGURE 9 – Latence p99 en fonction de l’utilisation ρ , avec le coude détecté par `elbow_helper.robust_knee` (ligne rouge pointillée et bulle) à $\rho \approx 0,605$.

loi de puissance, le taux de succès en fonction de la taille du cache C suit une forme de Michaelis-Menten, $C/(C + K)$, où K est la taille de cache à laquelle la moitié du jeu de données actif est déjà en mémoire : des gains rapides au début, puis une longue traîne qui s’aplatit. C’est concave et croissant, la même famille large que la courbe de saturation exponentielle deux sections plus haut, mais construite à partir d’une fonction rationnelle plutôt que d’une exponentielle, et répondant à une question de coût plutôt qu’à une question de constante de temps physique.

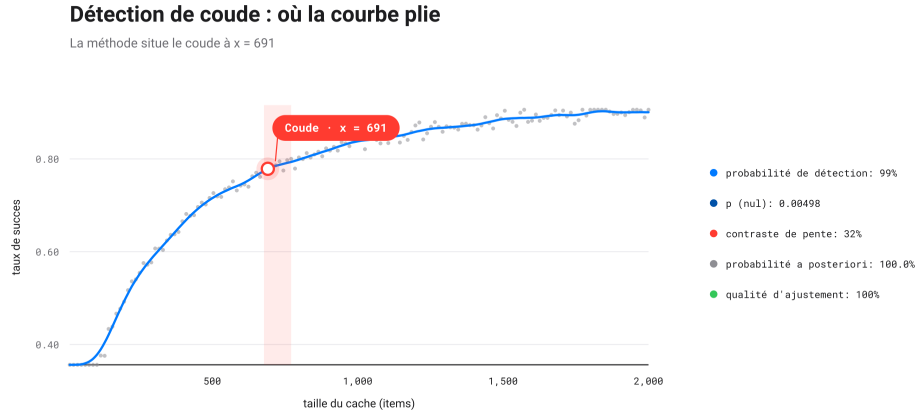


FIGURE 10 – Taux de succès en fonction de la taille du cache, avec le coude détecté par `elbow_helper.robust_knee` (ligne rouge pointillée et bulle) à 691 éléments, soit $3,46 \times K$.

Le coude se situe autour de $3,5K$, pas à K lui-même. K ne marque que le point de 50% de taux de succès, une propriété de l’algèbre de la formule, pas de l’endroit où le rendement *marginal* de la taille du cache cesse réellement d’être rentable. Acheter du cache jusqu’à K laisse beaucoup de taux de succès bon marché sur la table ; acheter au-delà de $3,5K$ environ dépense de la mémoire réelle pour une part de plus en plus mince de succès, exactement le chiffre dont une discussion de dimensionnement de capacité a besoin et dispose rarement sous forme close.

30 Un exemple travaillé : quand arrêter les tests

`examples/bug_discovery_rate.py` tourne la même famille convexe-décroissante que les exemples du k-means et de l'ACP vers une question d'ingénierie de release plutôt que de clustering ou de dimensionnalité. Les modèles de croissance de fiabilité décrivent la découverte de défauts pendant les tests comme rapide au début (les bogues faciles, à fort impact) puis lente (les bogues rares et profonds) : une courbe qui chute fortement, plie, puis s'aplatit en une longue traîne basse.

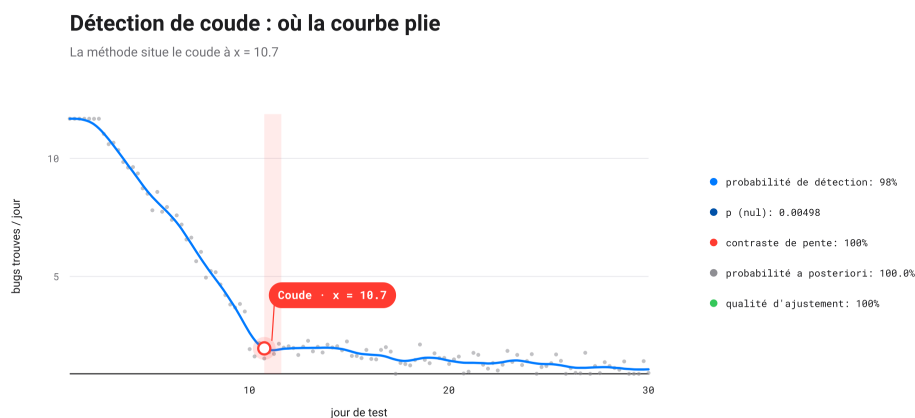


FIGURE 11 – Bogues trouvés par jour de test, avec le coude détecté par `elbow_helper.robust_knee` (ligne rouge pointillée et bulle) au jour 10,7, un jour après la vraie rupture au jour 10.

Le coude localisé se situe environ un jour après la vraie rupture, le même petit décalage systématique « un à deux points après la vraie pliure » que montre l'exemple du diagramme des éboulis de l'ACP plus haut. Lire `knee_x` comme « arrêter les tests exactement ici » plutôt que comme « le taux de découverte s'est maintenant vraiment aplati, à un jour ou deux près » accorde à l'estimation ponctuelle plus de confiance qu'elle n'en revendique réellement.

Pour aller plus loin

Pour un traitement plus approfondi ou plus rigoureux des idées ci-dessus que cette note n'en a l'ambition, non comme sources d'une affirmation précise déjà citée, mais comme pistes pour aller plus loin :

- Hastie, Tibshirani et Friedman sur les compromis biais-variance derrière le BIC, la validation croisée et le bootstrap (2009)
- Bishop sur le cadre bayésien de sélection de modèle qui motive le BIC et l'ICL (2006)
- Bouveyron, Celeux, Murphy et Raftery sur les modèles de mélange et le clustering à base de modèles, à l'échelle d'un ouvrage entier (2021)
- Cormen, Leiserson, Rivest et Stein sur la programmation dynamique en général (2022)
- López de Prado sur la rigueur de validation pour des données ordonnées et séquentielles (2018)

Références

- Christophe Biernacki, Gael Celeux, and Gerard Govaert. Assessing a mixture model for clustering with the integrated completed likelihood. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 22(7) :719–725, 2000.
- Christopher M. Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006.
- Charles Bouveyron, Gilles Celeux, T. Brendan Murphy, and Adrian E. Raftery. *Model-Based Clustering and Classification for Data Science*. Cambridge University Press, 2021.
- Alice Cleynen, Thanh Mai Luong, Guillem Rigai, and Gregory Nuel. Fast estimation of the icl criterion for change-point detection problems, with applications to next-generation sequencing data. *Computational Statistics and Data Analysis*, 2013.
- Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT Press, 4th edition, 2022.
- Sanjoy Dasgupta, Christos H. Papadimitriou, and Umesh V. Vazirani. *Algorithms*. McGraw-Hill, 2006.
- Robert B. Davies. Hypothesis testing when a nuisance parameter is present only under the alternative, 1977.
- Bradley Efron. Bootstrap methods : Another look at the jackknife. *The Annals of Statistics*, 7(1) : 1–26, 1979.
- Klaus Frick, Axel Munk, and Hannes Sieling. Multiscale change point inference. *Journal of the Royal Statistical Society : Series B*, 2014.
- Piotr Fryzlewicz. Wild binary segmentation for multiple change-point detection. *The Annals of Statistics*, 42(6) :2243–2281, 2014.
- Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning*. Springer, 2nd edition, 2009.
- Robert E. Kass and Adrian E. Raftery. Bayes factors. *Journal of the American Statistical Association*, 90(430) :773–795, 1995.
- Rebecca Killick, Paul Fearnhead, and Idris A. Eckley. Optimal detection of changepoints with a linear computational cost. *Journal of the American Statistical Association*, 107(500) :1590–1598, 2012.
- Marcos López de Prado. *Advances in Financial Machine Learning*. Wiley, 2018.
- Guillem Rigai, Emilie Lebarbier, and Stephane Robin. Exact posterior distributions over the segmentation space and model selection for multiple change-point detection problems. *Statistics and Computing*, 22(4) :917–929, 2012.
- Ville Satopaa, Jeannie Albrecht, David Irwin, and Barath Raghavan. Finding a kneedle in a haystack : Detecting knee points in system behavior. In *2011 31st International Conference on Distributed Computing Systems Workshops (ICDCSW)*, 2011.

- Gideon Schwarz. Estimating the dimension of a model. *The Annals of Statistics*, 6(2) :461–464, 1978.
- Pranab Kumar Sen. Estimates of the regression coefficient based on kendall’s tau. *Journal of the American Statistical Association*, 63(324) :1379–1389, 1968.
- Charles Spearman. The proof and measurement of association between two things. *American Journal of Psychology*, 15(1) :72–101, 1904.
- Henri Theil. A rank-invariant method of linear and polynomial regression analysis. *Proceedings of the Royal Netherlands Academy of Sciences*, 53, 1950.
- Yi-Ching Yao. Estimating the number of change-points via schwarz’ criterion. *Statistics and Probability Letters*, 6(3) :181–189, 1988.
- Nancy R. Zhang and David O. Siegmund. A modified bayes information criterion with applications to the analysis of comparative genomic hybridization data. *Biometrics*, 63(1) :22–32, 2007.